NPM Dependency Update Guide

STANDARD OPERATING PROCEDURE FOR NODE.JS APPLICATIONS

1 Pre-Update Safety Check

Never start updates on a "dirty" working directory. Ensure all current changes are committed to Git.

- **Git Status:** Run `git status` to ensure no pending changes.
- **Database Backup:** Export your Postgres data before updating libraries that interact with the DB (like `pg`).

Phase I: Identification

Before changing files, identify which packages have available updates:

```
npm outdated
```

Red: Update is within your version range (Safe).
Yellow: Update is a major version jump (Potentially Breaking).

Phase II: Safe Updates

Update packages to the latest version allowed by your `package.json` (typically minor/patch versions):

```
npm update
```

Apply security-specific patches identified by the NPM registry:

```
npm audit fix
```

Phase III: Major Version Updates

To move to versions beyond your current `package.json` constraints (e.g., v1.0 to v2.0), use **npm-check-updates**:

```
# Check updates and update package.json
npx npm-check-updates -u

# Install the new major versions
npm install
```

> ⚠️ **BREAKING CHANGES WARNING**
>
> Major version updates often remove features or change how a library is called. Always review the "Releases" or "Changelog" page on the library's GitHub repository before upgrading.

Phase IV: Deployment to Render

Render uses your `package-lock.json` to build your environment. You must push this file to trigger the update on the server.

```
git add package.json package-lock.json
git commit -m "chore: update dependencies"
git push origin main
```

Render Tip: If the build fails on Render after an update, use the "Clear Build Cache & Deploy" option in the Render dashboard to ensure no old package fragments are causing conflicts.